LANChat-H: Design, Implementation and Evaluation of a Dependency-Light Secure LAN Chat System with End-to-End Encryption

1st Harish Goud B
Dept. of Information Technology
Chaitanya Bharathi Institute of Technology
Hyderabad, India
harishgoudbit@cbit.ac.in

2nd Revanth Reddy Lingala
Dept. of Information Technology
Chaitanya Bharathi Institute of Technology
Hyderabad, India
revanthreddy0906@gmail.com

3rd Saarthak Manocha
Dept. of Information Technology
Chaitanya Bharathi Institute of Technology
Hyderabad, India
saarthakmanocha10@gmail.com

***Abstract*—Small teams and classrooms often want a private chat that stays on their own network, but the usual options are either heavy (federated protocols, brokers, web stacks) or simple demonstrations with no persistence, identity or protection from the network or from whoever runs the server. We present LANChat-H, a LAN chat system whose core uses only the Python standard library. It combines length-prefixed TCP framing with a plain-text fallback for older clients, a bounded outbound queue per client that isolates slow receivers, TLS with trust-on-first-use pinning, memory-hard credentials with lockout and rate limits, server-checked roles, SQLite persistence, ticketed file transfer and a hash-chained audit log. On top, end-to-end encryption (X25519, Ed25519, ChaCha20-Poly1305) covers private messages, files and an optional group room, so the server stores only ciphertext; the room uses one-way key chains per sender and epoch. We state the limits up front: there is no forward secrecy, the server still sees metadata, and trust is only as good as the comparison of security codes. On one laptop, repeated trials over loopback gave a median delivery latency for 100 receivers of 1.6 ms for a sequential relay, 2.3 ms for an `asyncio` relay and 6.9 ms for LANChat-H, which does more work per message; when one receiver stopped reading, a sequential relay delivered 2320 of about 5400 messages in every trial, while LANChat-H, like the `asyncio` relays, delivered all 6000; a 40-client mixed workload with file uploads over TLS, passed through an emulated network path, added about 7 ms on top of the path delay; and one core of the server host is saturated at about 200 receivers. Our own robustness testing (about 160,000 hostile events, a TLS probe and a static scan) found and fixed one real bug; it is not an independent audit. About 10,600 lines are covered by 453 tests and 370 scripted checks.**

***Index Terms*—LAN chat, end-to-end encryption, system design, message framing, head-of-line blocking, trust on first use, group messaging, audit log, Python**

I. Introduction

Small teams, classrooms and labs often want somewhere to talk and share files that stays on their own network. The mainstream answers are heavy: federated protocols such as XMPP, brokers such as MQTT, or web stacks with databases, containers and accounts held by third parties. The lightweight answers are the opposite: a few dozen lines of sockets that broadcast text, with no identity, no history, no protection from anyone on the network and none from whoever runs the server. Between the two there is little that a student, a teacher or a small office can read in an afternoon, start with one command, and still trust with private conversations.

Filling that gap exposes four problems that look separate but interact. First, TCP offers a byte stream, so a chat protocol must define its own message boundaries [2], [7], and that definition has to survive new features and older clients. Second, a server with many long-lived connections must not let one slow or stalled client delay the others, a head-of-line problem that sequential broadcast has by construction. Third, a local network is not a trust boundary: any host can connect, sniff and impersonate, so the system needs transport security, identity, throttling and an audit trail. Fourth, the person who runs the server should not automatically be able to read private conversations, which calls for end-to-end encryption in a system whose server also relays, stores, searches and moderates messages.

This paper presents LANChat-H, a chat system for local networks that addresses all four in one small code base whose core uses only the Python standard library [32], [33]. It began as a minimal threaded relay and grew through a sequence of milestones, each leaving earlier behaviour and the terminal client working (Section IV-H). What is new is the combination and its measured evaluation, not a new cipher or protocol (Section II-A). The contributions are:

1) a protocol and server design that keeps framing simple and compatible: length-prefixed frames carrying plain text or versioned JSON events, a renderer that lets text-only clients keep working as features are added, and a bounded outbound queue with a writer thread per client that isolates slow receivers;
2) a layered security design for a trusted-network deployment: TLS with trust-on-first-use pinning, memory-hard credentials with lockout and rate limits, roles checked on the server, ticket-authorised file transfer with hash verification, and a hash-chained audit log;
3) an end-to-end encryption layer for a server that also stores and relays: private messages and files, and an optional group room built on one-way per-sender key chains per epoch (late joiners read only what follows, removed members never receive the new keys), with

key rotation, recovery and an app lock, all from audited primitives;
4) an implementation of about 10,600 lines with one optional third-party package, covered by 453 automated tests and 370 scripted checks of the real window, including a test that scans every frame the server handles for the secrets of encrypted conversations;
5) a measured evaluation with repeated trials, variability, CPU and memory figures, comparison with sequential and `asyncio` relays, a mixed workload through an emulated network path, and our own robustness testing; and
6) an explicit statement of what the design does not protect: no forward secrecy, visible metadata, trust on first use, and measurements from a single machine.

Section II places the work among related systems. Section III states the goals and the threat model. Section IV describes the design, and Section V the implementation. Section VI evaluates it, Section VII discusses lessons and limits, and Section VIII concludes.

II. Background and Related Work

Transport and framing. TCP provides a reliable, ordered, full-duplex byte stream and nothing more [2], [1], [4], [5], [8], [9]: one `send()` need not correspond to one `recv()`, and functions that only the application can complete, such as message boundaries, cannot be delegated to the lower layer [7], [6]. Delimiter framing, as in the CRLF lines of IRC [18], is readable but needs an escaping rule; XML streams as in XMPP still need an outer mechanism to find where a unit ends [20]; WebSocket and MQTT carry lengths in binary headers [22], [23]. TCP keep-alive is optional and off by default [3], so noticing an unplugged peer needs application-level heartbeats. LANChat-H uses a four-byte network-order length [34], [10], which is binary-safe and exact at the price of validating the announced length.

Serving many clients. Thread-per-client gives each connection a sequential control flow that is easy to read and debug; event loops multiplex many connections on few threads but turn each into a state machine [17], [16]; staged designs such as SEDA join event-driven stages with bounded queues [12]; and the threads-versus-events debate is old and unresolved [13], [14], [15], [11]. Large systems isolate slow consumers with per-connection queues or non-blocking writes, but teaching-scale servers usually broadcast sequentially, so one stalled receiver delays everyone after it. LANChat-H keeps threads for readability and adds the isolation (Section IV-C); it does not claim that threads are the right choice at scale.

Messaging systems and secure messaging. IRC, XMPP, MQTT and WebSocket each solve interoperability and deployment problems (federation, brokers, browsers) that a single-server LAN chat does not have [18], [19], [20], [21], [23], [22]; Table I compares them with LANChat-H. TLS protects the channel [28], but the server still sees plaintext. End-to-end designs show how to hide content from a relaying server: the Double Ratchet [31], [30], X3DH for asynchronous contact set-up [49], and Messaging Layer Security for efficient groups [48], built on X25519, Ed25519, HKDF and ChaCha20-Poly1305 [44], [45], [47], [46]; Unger *et al.* compare the designs [29]. LANChat-H reuses the primitives and the sealed-key and sender-key ideas, not these full protocols.

Accounts, trust, audit, discovery and transfer. Authentication guidance recommends salted memory-hard hashing, checks against common passwords and throttling of repeated failures [50], [43]. SSH's trust-on-first-use host-key model is the precedent for self-signed certificates [40]. Hash-chained logs make alteration detectable, with a separate commitment needed to detect truncation [51], [52]. LAN services are normally found with IP multicast [24] and multicast DNS [41], [42], while small programs use a UDP broadcast, and FTP separated control and data connections [39], the pattern LANChat-H uses so that a large transfer never blocks the chat. Publish–subscribe and reliable group communication offer delivery guarantees far beyond what a single-server chat provides [25], [26], [27].

Each of these works treats one concern well. What is missing is a small artifact in which framing, isolation of slow clients, trust, persistence and end-to-end encryption can be read, tested and measured together. That is the niche LANChat-H occupies. It is a systems contribution, not a new protocol or cipher: the novelty is in how the pieces are combined and checked.

A. What Is New Here

Established secure messengers solve end-to-end encryption better than LANChat-H does: Signal's Double Ratchet gives forward secrecy and post-compromise security [31], [30], Matrix and OMEMO bring the same ideas to federated and XMPP deployments [36], [37], and MLS scales groups [48]. LANChat-H does not compete with them on cryptographic strength. Table II states the difference in what is offered. The contribution is a design point those systems do not occupy: a LAN-first system in one small readable code base, started with one command and no external services, in which the host is explicitly not trusted for private content, in which an administrator can still operate it (roles, audit log, backups, re-keying) and in which slow clients cannot stall the room, with each of these properties tested and measured in one place. Where the design is weaker (no ratchet, no multi-device, metadata visible) we say so in Section VII.

III. Design Goals and Threat Model

Goals. We set seven goals, each tied to something the evaluation checks.

G1 *Small and runnable.* Standard library for the core, one folder, one command, modules small enough to read.
G2 *Compatible growth.* New features must not break clients that predate them.
G3 *Isolation.* One slow, stalled or hostile client must not delay or crash the others.

TABLE I
COMPARISON OF RELATED APPROACHES WITH LANCHAT-H

Approach	Transport	Framing	Topology	Security	Persistence	Key limitation for a small teaching system
IRC [18], [19]	TCP	CRLF lines, 512-byte limit	Server network	None in base spec	None	Text-only framing; limits fixed by the spec
XMPP [20], [21]	TCP	XML stream	Federated client–server	SASL and TLS	Via extensions	Heavy parser; large specification surface
MQTT 5.0 [23]	TCP (optionally TLS)	Binary header, variable length	Broker pub/sub	Optional; credentials in CONNECT	Sessions and QoS	Needs a broker; more machinery than a chat room needs
WebSocket [22]	TCP after HTTP upgrade	Length-carrying frames; client masking	Client–server	TLS via `wss`	None	Needs an HTTP handshake and a framework
Signal / Double Ratchet [31], [30]	Any	Not specified	Relayed	End-to-end	Server-dependent	Cryptographic design, not a transport or server design
FTP [39]	TCP, separate control and data connections	Commands as text lines; data as a raw stream	Client–server	None in base spec	Files only	Predates TLS; no chat, presence or history
Flash [11]	TCP (web)	HTTP	Client–server	N/A	N/A	Web workload; compares process, thread and event models
SEDA [12]	TCP (services)	Application-defined	Staged pipeline	N/A	N/A	Targets far larger loads than a classroom chat
Threads vs. events [13], [14], [16]	TCP	N/A	N/A	N/A	N/A	Position papers, not chat-specific
Reliable groups [26], [27]	Reliable channels	N/A	Process groups	N/A	Varies	Guarantees far beyond a demonstration system
LANChat-H (this work)	**TCP + TLS**	**4-byte length + text or JSON events**	**Central relay**	**TLS with pinning, scrypt, roles, audit, end-to-end encryption**	**SQLite**	**Single server; no forward secrecy; metadata visible to the server (Sec. VII)**

TABLE II
QUALITATIVE COMPARISON WITH ESTABLISHED SECURE MESSAGING DESIGNS (FROM THEIR PUBLISHED DESIGNS, NOT FROM MEASUREMENTS OF THOSE PRODUCTS)

	Forward secrecy	Deployment	Group model
Signal [31]	Yes (ratchet)	Central service, internet and phone number	Sender keys, server-assisted
Matrix [36]	Yes (ratchet, Megolm sessions)	Federated; self-hosting is a separate server and database	Per-room sessions
XMPP + OMEMO [37]	Yes (ratchet)	XMPP server plus clients that implement the extension	Per-room device keys
IRC over TLS [18]	None end to end	One server, no content protection from it	Server relays plaintext
LANChat-H	**No** (stated limit)	**One folder, one command; the host is untrusted for private chats; audit log, roles**	**Per-sender chain per epoch; removal re-keys**

G4 *Safe on a hostile network.* Encryption in transit, a way to recognise the server, real accounts, throttling, and no file or name that can harm a recipient or the host.

G5 *Private from the host where it matters.* Private conversations, their files and (optionally) the group room must be unreadable to the server, with keys that survive a lost computer.

G6 *Accountable and operable.* Roles, an audit trail that shows tampering, administration tools, backups and upgrades that keep data.

G7 *Verifiable.* Every claim above is backed by a test or a measurement that can be rerun.

Assets. (i) The content of private messages, private files and, when enabled, the group room; (ii) the long-term identity keys and the passphrase and recovery code that protect them; (iii) accounts and their roles; (iv) the integrity of the history and of the audit log; (v) the availability of the room for everyone else.

Adversaries and assumptions. Table III lists whom we defend against. We assume the clients' computers are not compromised while a window is unlocked, that cryptographic primitives and the operating system's random generator are sound, and that the people who compare security codes do so over a channel the server does not control. We do not try to hide metadata, to defend a compromised endpoint, to resist a global network observer, or to scale beyond tens of people.

Key compromise, recovery and identity verification. Table IV walks through what happens when a key or a secret is lost or exposed, since these cases decide how far the guarantees hold.

TABLE III
ADVERSARIES AND WHETHER THEY ARE DEFENDED AGAINST

Adversary	Outcome
Another host on the network	Defended: TLS, certificate pinning, lockout, rate limits
A member beyond their role	Defended: roles checked on the server; hostile files and names neutralised; replays and forged senders detected
A malformed or hostile client	Defended: size-limited frames, validated events, one bad request refused without dropping others (Section VI)
The host or a thief of its disk	Partly: private messages, private files and the optional room are ciphertext; metadata and anything sent unencrypted are not
A server that lies about keys	Detected, not prevented: pinning and safety numbers
A stolen, unlocked computer	Not defended; auto-lock limits exposure

TABLE IV
WHAT HAPPENS WHEN A KEY OR SECRET IS LOST OR EXPOSED

Event	Consequence and remedy
Key file copied from a disk	The attacker must guess the passphrase offline; each guess costs one scrypt run (about 60 ms here), so a weak passphrase falls and a strong one holds. Rotate keys; everything that person could read before remains readable to the thief
Device stolen unlocked	All that person's decrypted history and keys are exposed; rotate and re-key rooms. The app lock reduces the window only
Passphrase forgotten	Restore from the recovery code (about 119 bits) on any computer
Passphrase and recovery code lost	Old encrypted messages are unreadable by design; a reset publishes a new identity and contacts see a changed-key warning
Long-term key stolen later	No forward secrecy: ciphertext already held by an attacker can be opened. A ratchet is future work
Server swaps someone's key	First contact: not detectable without comparing the 60-digit safety numbers. After pinning: sending is blocked until the new key is accepted
Member removed from the room	A new epoch starts and the removed person never receives its keys; what they already read stays read

IV. SYSTEM DESIGN

A. Architecture

Fig. 1 shows the structure. The server owns membership, persistence and policy; clients are thin. Both client kinds hold one TLS connection to a central relay, and file bytes use a second, short-lived connection authorised by a ticket from the first. A UDP responder lets clients find servers on the same network.

B. Framing, Protocol Versions and Compatibility

Every message in both directions is a four-byte big-endian length followed by UTF-8 text, as in Listing 1; zero-length and over-long frames close the connection. Two protocol versions share this framing. Version 1 carries plain text lines and is what the terminal client speaks. Version 2 carries one JSON object per frame with a type field (message, edit, delete, reaction, typing, presence, read marker, pin, search, history page, upload, download, key and room events, and so on); a version-2 client announces itself with a JSON hello in the username step, and the server detects the version from the

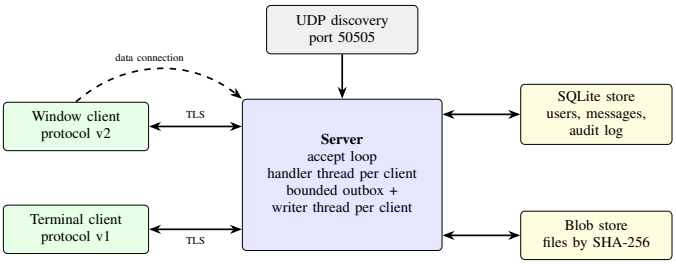


Fig. 1. Architecture of LANChat-H. Both client kinds hold one TLS connection to a central relay; file bytes use a second, short-lived data connection authorised by a ticket from the first.

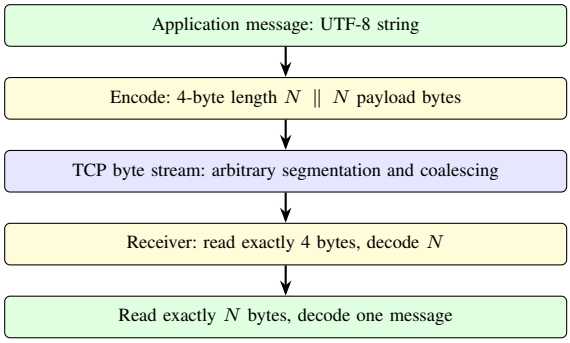


Fig. 2. Length-prefixed framing over a TCP byte stream.

first character. For version-1 clients the server renders every event as the line they have always seen, and sends nothing for events a terminal cannot show, so features were added without breaking the older client (G2). Message text is limited to 3400 bytes so that the JSON envelope always fits one frame; encrypted private messages are limited to 1500 bytes of text, so that the sealed keys and signature still fit in one frame.

C. Concurrency and Delivery

Each connection has a handler thread that reads requests and a writer thread that is the only code writing to that socket. Delivering a message means placing it on every recipient's bounded outbound queue (default 100 entries) without waiting. If a queue fills, the client is not keeping up and is disconnected; it can reconnect and replay history from the database (G3). The welcome and history are written directly before the writer thread starts, so they arrive first and in order. Clients send a ping every ten seconds and the server drops a connection silent for thirty. A user name may be signed in once; a saved sign-in may take over a stale connection. Section VI measures the effect of the queues.

D. Persistence and Features

Messages, users, reactions, pins, read markers, sessions, public keys and the audit log live in SQLite, so history survives restarts; seven numbered migrations let an older database upgrade in place (G6). A joining client receives the last 50 messages and can page back 50 at a time. Search uses an FTS5 index and falls back to `LIKE` where FTS5 is absent. The window supports replies, reactions, edit and delete

```
MAX_MESSAGE_BYTES = 4096

def send_message(conn, message):
    payload = message.encode("utf-8")
    if len(payload) > MAX_MESSAGE_BYTES:
        raise ValueError("message is too large")
    conn.sendall(struct.pack("!I", len(payload)) + payload)

def receive_message(conn):
    header = _receive_exactly(conn, 4)
    if header is None:
        return None            # peer closed
    (length,) = struct.unpack("!I", header)
    if length == 0 or length > MAX_MESSAGE_BYTES:
        raise ValueError("invalid message length")
    payload = _receive_exactly(conn, length)
    if payload is None:
        return None
    return payload.decode("utf-8")
```

Listing 1. Framing logic of the reference implementation (`protocol.py`, condensed).

(deletion keeps a tombstone so quoted replies still resolve), typing indicators, presence, read receipts, mentions, pins and an administrator console. These features stress the protocol: history paging and search need replies that respect back-pressure, edits and deletes need ids that survive reconnection, and, as Section IV-G shows, search, pins and mentions need rethinking when the server cannot read the text.

E. Bulk Transfer

Pictures, documents and audio do not travel on the chat connection. The client sends an upload request with the name, size and SHA-256; the server checks role, type, size, quota and conversation and replies with a one-time ticket. The client opens a second TLS connection, presents the ticket, and streams raw bytes; the server writes them to a temporary file while hashing, enforces the declared size and a deadline, compares the digest, and only then moves the file into place under its hash and posts the message. Downloads mirror this and are verified by the receiver; a damaged file is never delivered, and a half-finished upload leaves nothing behind. Identical content is stored once, and a file is removed when the last message referring to it is deleted. File names are reduced to a safe base name and executable types are blocked (G4).

F. Transport and Account Security

Table V lists each threat with its control; the controls are in code and covered by tests (Section VI). Together they implement G4 and the non-confidentiality parts of G6; the confidentiality controls of G5 are described next.

G. End-to-End Encryption

End-to-end encryption is the part of the design that changes what the server is. Without it the server decrypts TLS, stores and relays plaintext and can search it; with it the server stores ciphertext and public keys, and everything that needs the words moves to the client.

Identities and trust. Each person has an X25519 key pair for key agreement and an Ed25519 pair for signing, bound by a self-signature [44], [45]. Secret keys are wrapped on disk with ChaCha20-Poly1305 [46] under a key derived by scrypt from a passphrase. A 24-character recovery code (about 119 bits) wraps a second copy that the server stores as an opaque backup; because the code is high entropy the server cannot usefully guess it, and a test confirms that neither the keys nor the code appear in the database file. The server keeps only public keys and cannot be trusted to report them honestly, so clients verify each bundle's signature, pin the first key seen for each contact, block sending when it changes until the person has accepted the new one, and offer a sixty-digit safety number computed from both identities for comparison in person or on a trusted call.

Private messages. The sender encrypts the text once under a random key with ChaCha20-Poly1305, seals that key separately for the recipient and for themselves (ephemeral and static X25519 through HKDF-SHA256 [47]), and signs the whole message with Ed25519, binding the sender and recipients into the associated data. The server checks only the shape and the two names, stores an empty body and the envelope, and relays it. Tampering, a wrong recipient or a forged sender fails to open, and the window flags a reused random message id as a possible replay. Encrypting for the sender as well as the recipient costs a few hundred bytes and lets people read their own history after a restart.

Files. Files are encrypted on the sender's computer in 64 KiB chunks under a random key; each nonce carries the chunk number and a last-chunk flag, so dropped, reordered or added chunks fail. The key, the real name and the type travel inside the encrypted message, so the server stores unreadable bytes under an opaque name and can apply only size and space limits; the receiving window, which alone knows the name, warns before opening a program-like file.

The group room. A sealed copy for every member does not scale to a room, so the room uses sender keys in the style of group messengers [48]. Each member holds a random chain key per *epoch*, derives the key for message n by n one-way steps, and gives every other member a sealed copy of the chain as it stands, so a member who joins later reads only what follows. The server stores the sealed copies (so they can be fetched again after a restart), enforces the current epoch and strictly increasing message numbers per sender, and starts a new epoch whenever someone is removed or an administrator asks; the removed person never receives the new chain. Mentions are named by the sender because the server cannot read the text, and `@everyone` stays administrator-only; pins, quoted replies and search use what the window has decrypted. An administrator switches the room on, after seeing which members have no keys, because people without keys and terminal clients can then neither read nor write.

Key lifecycle. Keys can be rotated (a new identity, with the old ones kept so earlier messages still open and the server keeping the envelopes sealed to them), restored from the recovery code on a new computer with every identity, or reset, which discards history. An idle window locks itself, covering the interface, hiding dialogs and forgetting the keys until the passphrase is entered.

What the server still learns, and what is not provided.

TABLE V
THREATS AND CONTROLS IN LANCHAT-H

Threat	Control
Reading or altering traffic	TLS (1.2 or newer) on chat and data connections. The host creates a self-signed certificate once.
Impersonating the server	Trust on first use: the client shows the certificate's SHA-256 fingerprint, pins the certificate after approval and refuses a different one later. Host name checking is off because LAN addresses change; the pinned certificate authenticates the server.
Stolen account database	Passwords stored only as salted scrypt hashes ($n = 2^{14}$, $r = 8$, $p = 1$, 32-byte output) with constant-time comparison.
Password guessing	Five wrong passwords lock a name from an address for 30 s, doubling to 15 min; one address failing for many names is locked after 20; a correct password clears the count. New passwords need six characters, must differ from the user name and common passwords; at most ten sign-ups per address per hour.
Flooding	Per-person token bucket (30 burst, 10 per second); 240 new connections per address per minute; frame, message and file size limits.
Privilege abuse	One permission table; every action checked on the server; the last admin cannot be demoted or disabled.
Password kept in memory	After sign-in the window keeps a random token (12 h), stored on the server only as a SHA-256 hash, revoked on leave, password change, disable or reset.
Hostile files	Names reduced to a base name; files stored under their SHA-256; executable types blocked; size and space limits enforced while streaming; hash checked on upload and on download; tickets are single-use, expire in 60 s and are limited per person.
Disputed history	Audit log of sign-ins, failures, lockouts, role changes, deletions of others' messages, uploads, resets and settings, each row hashed with its predecessor; a head marker detects truncation; message text is never logged.
The host reading chats and files	End-to-end encryption of private messages, files and (optionally) the group room: the server stores ciphertext and public keys only (Sec. V-G).
A fake key from the server	Self-signed key bundles, pinning on first sight, sending blocked after a change until accepted, sixty-digit safety numbers.
Replays and forged senders	Every encrypted message is signed and bound to its sender and recipients (or epoch and number); the window flags reused ids and numbers.
A removed member	A new key epoch for the room, whose keys the removed person never receives.
An unattended window	Auto-lock after an idle time covers the window and forgets the keys.

The server sees who talks to whom, when, the sizes of messages and files, reactions, who is mentioned, each person's role and the membership of the room; it can also withhold or delete messages, though it cannot read or forge them. There is *no forward secrecy* (Table IV); identity is only as strong as the out-of-band comparison of safety numbers; and there is one active device per person. These limits are repeated in the abstract and in Section VII because they bound every confidentiality claim in this paper.

H. Development Path

LANChat-H was built in the milestones of Table VI. Each ended with the whole test suite and the scripted window runs passing, and with the terminal client unchanged in behaviour (G2). The order matters: persistence and a versioned protocol came before the features that need message ids, security hardening before file sharing opened new attack surface, and end-to-end encryption last because it depends on identities, a key directory, the ticketed transfer path and the event schema all being stable.

TABLE VI
MILESTONES OF THE IMPLEMENTATION

Milestone	What was added
Baseline	Threaded relay, four-byte framing, plaintext broadcast, three tests
Robustness	Bounded queue and writer per client, heartbeat, unique names, TLS with pinning, accounts
Persistence and roles	SQLite, migrations, history, admin/member/guest roles
Social features	JSON events with a text fallback; replies, reactions, edits, presence, read receipts, search, mentions, pins, history paging
Hardening	Lockout, rate limits, password rules, saved sign-ins, password change, data export
Media	Ticketed file transfer, inline pictures, audio, hash verification
Administration	Admin console and CLI, hash-chained audit log
E2EE 1: keys	Identities, key directory, recovery code, safety numbers, pinning
E2EE 2: messages	Encrypted private messages, replay and forgery checks, policy
E2EE 3: files	Chunked file encryption, opaque server storage
E2EE 4: room	Per-sender key chains, epochs, late joiners, removal re-key
E2EE 5: hardening	Key rotation, app lock, wire-scan test, documentation

V. IMPLEMENTATION

The core uses only the Python standard library [32], [33], [34], [35]: `sqlite3` and `hashlib` supply storage and scrypt [53], [54], and `tkinter` supplies the desktop window. Three third-party packages are optional: `cryptography` for end-to-end encryption, Pillow for some picture previews and `sounddevice` for voice recording; without the first the chat works over TLS only and the window says so (G1). In total the non-graphical code is about 4,800 lines in 21 modules, the window about 4,900 lines in seven modules, and the end-to-end encryption code about 900 lines in three modules. Table VII maps the modules to the design. The tests start real servers on free ports, and scripted drivers open the real window and click through it; the benchmarks of Section VI are in the repository [55].

VI. EVALUATION

All measurements were taken on one machine: an Apple-silicon laptop (arm64) running macOS 27.2 with CPython 3.14.9, with the server and clients on the loopback interface,

TABLE VII
IMPLEMENTATION MAPPING AND HONEST SCOPE

Component	Concept	Role in the system	Boundary / not provided
`protocol.py`, `events.py`	Record recovery; typed events	4-byte network-order length, exact-read loop, 4096-byte limit; JSON events with a type field, plus a renderer that turns any event into a plain line for old clients	No acknowledgements or sequence numbers on the wire
`server.py`, `server_admin.py`, `server_keys.py`, `server_room.py`	Accept, membership, fan-out, policy	Handler thread and bounded outbox with writer thread per client; handshake; history, search, pins; ticketed file transfer; admin and key-directory events	Single server; no failover; one thread per client
`store.py`, `accounts.py`, `permissions.py`	Persistence, credentials, roles	SQLite with six numbered migrations, FTS5 search with `LIKE` fallback; scrypt hashes; admin, member and guest roles checked on the server	One connection behind a lock; not a clustered database
`security.py`, `audit.py`	Abuse resistance; accountability	Lockout with backoff, token-bucket rate limits, connection and sign-up caps, password rules; hash-chained audit log	In-memory counters reset on restart
`storage.py`, `transfer.py`	Bulk transfer	Content-addressed blob store, name sanitising, blocked types, streaming SHA-256 verification	Files are not encrypted at rest on the host
`chat_core.py`, `certs.py`, `discovery.py`	Client handshake, TLS trust, discovery	Login, certificate pinning by SHA-256 fingerprint, self-signed certificate creation, UDP broadcast discovery	Discovery works within one broadcast domain
`e2ee.py`, `e2ee_group.py`, `keystore.py`	End-to-end encryption	Identities, passphrase-wrapped key file, recovery code, safety numbers, pinned contacts, sealed messages and files, the room's key chains	No forward secrecy; the server still sees metadata (Sec. VII)
`gui_*.py`, `client.py`, `chat.py`, `admin_cli.py`	Interface	Desktop window, terminal client, single launcher, offline admin tool	Terminal client lacks edits, reactions, search and pictures

and the server in one process and the load generators in separate processes. The source code, tests, scripts and the raw results (`bench/`, `bench/results/`, `security_tests/`) are published [55]. Because client and server share one machine, the results show the cost of the software, not of a real network (Section VI-D adds an emulated path). Unless stated, each setting was repeated and we report the mean $\pm$ standard deviation.

A. Correctness Tests

The repository holds 22 test modules that run 453 tests, each starting real servers on free local ports; one is skipped where a platform feature is missing. They cover framing (round trip, closed connection, oversized and zero-length frames), the version-1 and version-2 protocols, the database and its migrations, search and paging, TLS and certificate pinning including a changed certificate, discovery, file transfer (round trips, hash mismatch, cut-off uploads, hostile names, expired and reused tickets, quota, concurrent uploads), lockout, rate limits, password rules, saved sign-ins, administrator actions, tamper detection in the audit chain, and the encryption code, which is checked against published test vectors for X25519 and Ed25519 [44], [45] and then against tampering, wrong recipients, forged senders, replays, damaged and reordered files, and the room's rules for late joiners, removal and message numbering. A further eleven scripts drive the real window and the terminal client through 370 checks, including runs with every security default on.

B. Fan-Out Latency and Comparison with Other Servers

One sender sent 60 messages of about 100 bytes at ten per second to N receivers, each a real client with its own connection; latency is the arrival time minus the send time on the machine's monotonic clock. To compare with other designs we wrote three minimal relays that speak the same wire protocol: a *sequential relay* (thread per client, blocking send to each recipient in turn), an *asyncio relay* (one event loop, a bounded queue and a writer task per client) and an *asyncio relay with SQLite*, which stores every message before delivering it. None of them checks accounts, roles or rate limits or renders events for two protocol versions, so they show a lower bound for each design, not a like-for-like product. Peak memory and CPU time belong to the server process and were measured in a fresh process for each trial.

Table VIII shows that every message reached every receiver in every trial, and that the spread between trials is small (a standard deviation of at most 0.4 ms for the median). The medians for LANChat-H rise from 4.3 ms with 10 receivers to 6.9 ms with 100, against 1.0 to 2.3 ms for the `asyncio` relay and 2.0 to 3.0 ms with SQLite. The remaining difference is the work that LANChat-H does for each message: checking the sender's role and rate limit, storing the message, parsing mentions, and rendering it for each protocol version. The same cost shows in CPU time: 4.4 s over a ten-second run with 100 receivers, against 0.2 s for the `asyncio` relays, and 54 MB of memory against 33 MB. TLS added about 0.4 ms at 100 receivers. At 200 receivers the server used 9.6 s of CPU in a run of about ten seconds, so the design saturates one core, as its

single-process Python threads would suggest, at roughly 200 receivers sending ten messages a second; this is the point at which an `asyncio` or multi-process design would be needed, and we do not claim the thread-per-client server scales beyond it.

C. Head-of-Line Blocking

The fan-out cost model predicts the effect of a stalled receiver. With sequential sends, recipient k receives the message after

$$D_k = \sum_{j=1}^{k} s_j, \qquad T_{\text{bcast}} = D_N, \tag{1}$$

where s_j is the time the blocking send to recipient j takes, and if recipient m stalls for an extra time W then

$$D_k \;\geq\; W + \sum_{j=1}^{k} s_j \qquad \text{for all } k \geq m. \tag{2}$$

With a bounded queue and writer thread the send to each recipient reduces to an enqueue with cost $q_j \ll W$, so D_k no longer depends on W for $k \neq m$.

Ten receivers read normally; one more connects first, shrinks its receive buffer to 4 KiB and never reads. A sender sends 600 messages of 2000 bytes at 50 per second. Once the kernel buffers for the stalled connection were full, after 232 messages (about 460 kB), the sequential relay stopped delivering to every receiver: exactly 2320 deliveries arrived in all ten trials, out of 5370 to 5400 expected, and the sender's own sends began to time out. LANChat-H and both `asyncio` relays delivered 6000 of 6000 in all ten trials (Table IX), so the isolation comes from the bounded queue, not from threads or from our code in particular: any design that never blocks on one receiver has it. LANChat-H also dropped the stalled connection once its queue filled in all ten trials. In the absence of a stall the sequential relay is faster per message (under 1 ms against 4 ms) for the reasons given above, so isolation is a trade of a few milliseconds for the guarantee that one bad connection cannot stop the room.

D. Mixed Workload and Emulated Network Path

Fan-out at a regular rate is not how people chat, so we also ran a mixed workload with every default limit on and TLS enabled: 40 clients (100 in the last row), each sending a text message after an exponentially distributed pause (mean 6 s) with a log-normal size (median about 60 bytes), every client sending the heartbeat that real clients send, and one more client uploading a 1 MB file every 5 seconds while the chat ran. To emulate a network path on one machine we put a small user-space proxy between clients and server that delays each direction by a base time plus Gaussian jitter (`bench/netem.py`). It cannot create packet loss, which TCP hides from applications as extra delay, so loss is represented only by jitter.

Table X shows that every message reached every receiver in every run. With no added delay the median stayed at about 5 ms with 40 clients and about 7 ms with 100, and the 1 MB uploads, which ran alongside the chat, finished in about 20 ms. With an added one-way delay d the median grew to about $2d + 7$ ms (d=5: 17 ms; d=20: 47 ms; d=50: 108 ms; d=100: 207 ms), that is, the server added a roughly constant 7–8 ms on top of the two crossings of the path, and the upload time grew in line with the round trips that the ticket exchange needs. This is an emulation, not a measurement of Wi-Fi or Ethernet; the script `bench/lan_probe.py` measures the same round trip from a second device against a real server, but we have not yet run it across several machines.

E. File Transfer Speed

Table XI gives the median and range of seven runs for uploading and then downloading a random file through the ticket and data-connection path, including hashing, the ticket exchange and verification. Throughput is in the hundreds of megabytes per second on loopback, rises with file size, and TLS costs roughly a third for small files and less for large ones. The runs vary most for 1 MB files, where fixed costs dominate. These figures show that the design adds no obvious bottleneck; they say nothing about a wireless link, which would dominate in practice.

F. Cost of Security Operations

Table XII lists the median time of single operations over 30 runs; the standard deviation was under 10 % of the median for every operation. A password check costs about 29 ms by design: it is the price of making offline guessing expensive. Encrypting a message for one person (a random key sealed twice and a signature) takes about 0.3 ms and decrypting it 0.25 ms, a room message under 0.3 ms, and a 20 MB file is encrypted in about 30 ms and decrypted in about 20 ms (roughly 670 and 990 MB/s). The cost of end-to-end encryption is therefore dominated by passphrase stretching and key management, not by the ciphers, and the server's own work is unchanged: it relays the same frames.

G. Confidentiality Check

The confidentiality claim of G5 is checked by observation rather than by argument. The test wraps the server's receive and send functions so that every frame it handles is recorded. It then has two people exchange a private message, a private edit, a room message and an encrypted file with a caption, and scans all recorded frames, the SQLite file and the stored files for the secret message texts, the real file name and type, the file contents, the file key and the people's secret keys. None appears; the traffic does contain the encrypted envelopes, so the capture is real. A companion check sends a plaintext message through the same capture and finds it, so the scan would notice a leak. Separate tests assert that the database holds an empty body and an opaque attachment name for encrypted messages, that neither a chain key nor a recovery code is stored, and that a removed member, a late joiner and a person without keys cannot open what they should not.

TABLE VIII
BROADCAST LATENCY, CPU AND MEMORY BY SERVER AND NUMBER OF RECEIVERS (LOOPBACK; MEAN ± STANDARD DEVIATION OVER 8 TRIALS, 5 FOR TLS AND 200 RECEIVERS; EVERY MESSAGE REACHED EVERY RECEIVER IN EVERY TRIAL)

Receivers	Server	Median (ms)	p95 (ms)	CPU (s)	Peak RSS (MB)
10	Sequential relay	0.67 ±0.03	1.16 ±0.06	0.02 ±0.00	32.2 ±0.3
	asyncio relay	1.01 ±0.08	1.62 ±0.17	0.05 ±0.00	32.4 ±0.3
	asyncio + SQLite	1.99 ±0.05	2.83 ±0.12	0.09 ±0.00	33.4 ±0.2
	LANChat-H	4.34 ±0.07	5.50 ±0.37	0.61 ±0.01	47.3 ±0.2
50	Sequential relay	1.05 ±0.09	2.25 ±0.11	0.06 ±0.00	33.7 ±0.3
	asyncio relay	1.72 ±0.03	3.23 ±0.19	0.13 ±0.00	33.0 ±0.2
	asyncio + SQLite	2.53 ±0.21	4.11 ±0.38	0.15 ±0.02	34.0 ±0.2
	LANChat-H	5.78 ±0.12	7.19 ±0.32	2.24 ±0.04	50.6 ±0.2
100	Sequential relay	1.60 ±0.19	3.35 ±0.25	0.11 ±0.01	35.4 ±0.3
	asyncio relay	2.32 ±0.13	4.40 ±0.28	0.21 ±0.01	33.0 ±0.2
	asyncio + SQLite	2.98 ±0.22	4.75 ±0.43	0.24 ±0.01	33.8 ±0.2
	LANChat-H	6.89 ±0.18	8.84 ±0.25	4.40 ±0.25	54.0 ±0.2
100 (TLS)	**LANChat-H**	7.32 ±0.09	9.68 ±0.51	4.48 ±0.07	59.4 ±0.1
200	**LANChat-H**	10.14 ±0.37	14.39 ±1.77	9.56 ±0.15	62.1 ±0.2

TABLE IX
ONE STALLED RECEIVER, TEN HEALTHY ONES (10 TRIALS PER SERVER; 600 MESSAGES OF 2000 BYTES AT 50 PER SECOND; DELIVERIES TO THE TEN HEALTHY RECEIVERS)

Server	Delivered	Median (ms)	Stalled dropped
Sequential relay	2320/5370–5400	0.66 ±0.06	not measured
asyncio relay	6000/6000	1.01 ±0.05	not measured
asyncio + SQLite	6000/6000	1.56 ±0.09	not measured
LANChat-H	6000/6000	4.01 ±0.04	10/10

TABLE X
MIXED WORKLOAD OVER TLS WITH ALL LIMITS ON (60 S; TEXT MESSAGES AT RANDOM TIMES PLUS A 1 MB UPLOAD EVERY 5 S; EMULATED ONE-WAY DELAY ± JITTER IN MS)

Cl.	Delay	Med.	p95	p99	Deliv.	1 MB (s)	CPU (s)
40	0 ± 0	4.8	6.7	7.9	100.0 %	0.02	4.2
40	5 ± 1	16.7	20.5	22.1	100.0 %	0.08	4.2
40	20 ± 4	47.0	57.0	61.5	100.0 %	0.24	4.3
40	50 ± 10	108.0	131.7	141.8	100.0 %	0.57	4.3
40	100 ± 20	207.4	253.8	272.6	100.0 %	1.08	4.4
100	0 ± 0	6.8	10.6	14.1	100.0 %	0.02	15.9

TABLE XI
FILE TRANSFER THROUGHPUT (LOOPBACK, MB/S, MEDIAN [RANGE] OF 7 RUNS)

File (MB)	TLS	Upload	Download
1	no	273 [143–318]	573 [425–703]
1	yes	185 [160–194]	302 [284–337]
5	no	660 [571–702]	1032 [945–1127]
5	yes	418 [393–476]	648 [593–696]
20	no	816 [732–872]	950 [892–1085]
20	yes	636 [572–653]	691 [548–746]

TABLE XII
COST OF SINGLE OPERATIONS (MEDIAN OF 30 RUNS)

Operation	Time (ms)
Password check (scrypt $n = 2^{14}$)	28.6
Wrap secret keys with a passphrase (scrypt $n = 2^{15}$)	59.6
Generate an identity (X25519 and Ed25519)	0.05
Safety number (5200 SHA-512 rounds)	2.1
Encrypt a 200-byte message for one person	0.30
Decrypt it	0.25
Room message: encrypt / decrypt	0.16 / 0.25
Seal a chain key for one member	0.26
Walk a room chain forward 1000 steps	1.4
Encrypt / decrypt a 20 MB file	29.9 / 20.2

H. Our Own Robustness and Security Testing

We also tested the server as an attacker would, with tools we wrote or ran ourselves. *This is not independent testing*; an external review remains future work and is listed as a limit.

- **Fuzzing** (`security_tests/fuzz.py`). Twenty seeded runs against a live server sent about 29,600 hostile connections before sign-in (random bytes, zero and huge length prefixes, truncated frames, malformed hellos, HTTP requests, oversized passwords), about 160,000 mutated events after sign-in as a member and as an administrator, covering every event type the server knows with fields replaced by hostile values (null, huge integers, long strings, nested containers, wrong types), and 4,000 made-up transfer tickets. After each run a fresh client still signed in and exchanged a message. The first version of this test found one real bug: an integer above 2^{63} in a field that reaches SQLite raised an unhandled `OverflowError` that ended the sender's connection thread. We fixed it by refusing the malformed request while keeping the connection, added a regression test, and the final runs found no unhandled exception in any server thread, with memory growth of about 1 MB.
- **TLS probe** (`security_tests/tls_probe.py`). The server refused TLS 1.0 and 1.1, accepted 1.2 (ECDHE-RSA-AES256-GCM-SHA384) and 1.3 (TLS_AES_256_GCM_SHA384), refused a client pinned to a different certificate and a client trusting only public authorities, and reset a plain-text client.

- **Static scan.** Bandit reported no high-severity finding in the code (about 13,600 lines scanned, tests included). Of the 10 medium findings in the server and client code, 3 are the intended bind to all interfaces of a LAN server and 7 are queries built from fixed fragments and question-mark placeholders only; we inspected each and found no injection path.

I. Threats to Validity

- *One machine, one Python version, loopback.* Client and server compete for the same cores, and no real network, wireless loss or second device was involved; the emulated delay shows the behaviour of the server under a slow path, not of a real one. A real multi-device study was prepared (`bench/lan_probe.py`) but not run.
- *Few trials.* Eight trials per setting (five for TLS and for 200 receivers, ten for the stalled-receiver test) give a tight spread here but are not a statistical study; the mixed workload was run once per setting, and the 100-client row should be read as indicative.
- *Baselines are minimal relays*, not other chat products. They show what the same wire protocol costs with and without the extra work of LANChat-H, and they are not measurements of Signal, Matrix, XMPP or IRC servers; no suitable open implementation with the same features and protocol was available for a like-for-like comparison.
- *Synthetic load.* The workload is generated from simple distributions, not a trace of real conversations.
- *Self-run security testing.* The fuzzing, TLS probe and static scan were written and run by the authors, with a fixed set of generators; they show that these inputs are handled, not that no other input breaks the system, and they are not a substitute for an independent review.

VII. Discussion and Limitations

Lessons. A renderer that turns every event into the line an older client expects made incremental growth cheap: twelve milestones later the terminal client still works, for plain conversations. LANChat-H's per-message work (persistence, checks, rendering and the queues) costs a few milliseconds more than a bare relay in our measurements, but the queues remove an entire class of failure, and writing the slow-receiver experiment forced us to say precisely what "isolation" means. Testing against real servers and the real window found problems that unit tests of pieces would not: malformed key data that raised an unexpected exception in the key-handling code, a recovery-code "Done" button that was packed behind a sibling widget and so invisible, and an ordering bug in which a test thread waited on the server while the window loop it depended on was not running. Finally, end-to-end encryption is less about the cipher than about everything the server used to do with plaintext: search, pins, mentions, quoting, policy checks and moderation each needed a client-side or metadata-based replacement.

No forward secrecy. Private messages use an ephemeral key per message, which protects the sender's side, but theft of a recipient's long-term key would expose earlier messages to anyone holding the ciphertext, and the room's sealed chain copies stay on the server so that history can be re-read. A ratchet [31], [30] would be the upgrade, at the cost of storing per-device state.

Identity is as strong as the comparison. Keys are pinned on first sight and a change blocks sending, but until two people compare security codes a hostile server could have substituted a key at the very first contact. The same holds for the server certificate under trust on first use.

Metadata and moderation. The server learns who talks to whom and when, sizes and mentions. It cannot read encrypted content, so administrators cannot moderate by reading; they can delete by id, remove people and re-key.

Scope of encryption. Terminal clients and members without keys cannot take part in encrypted conversations; search of encrypted messages covers what the window has loaded; and files shared in readable conversations are stored unencrypted. The window is not a vault: decrypted text lives in memory while it is open.

Scale and evidence. The design is one server, one thread per client and one SQLite connection behind a lock. Our measurements put the single-core limit near 200 receivers at ten messages a second, where a design on an event loop [38] would use far less CPU for the same fan-out (Table VIII); we did not build a full `asyncio` version of LANChat-H, so how much of its feature cost would remain is untested. A study across several real devices and wireless links is also still to be done. The security claims rest on tests of this implementation, our own fuzzing and a static scan, not on a formal analysis or an independent review, and the primitives are audited but their composition here is new.

Future work. A ratchet for forward secrecy, multiple devices per person, sealed-sender style protection of metadata, an asynchronous server with the full feature set, measurements over real Wi-Fi and Ethernet with several devices, and an independent security review.

VIII. Conclusion

LANChat-H shows that a chat system for a local network can combine a simple compatible protocol, isolation of slow clients, layered transport and account security, and end-to-end encryption for private conversations and an optional group room, in a code base small enough to read and with one optional dependency. The measurements on one machine show what the design costs and buys: a few milliseconds of extra per-message work (storage, checks and queueing; about 7 ms on top of the path delay in our mixed workload) in return for a room that survives a stalled receiver, and a fraction of a millisecond per encrypted message in return for a server that holds only ciphertext. A test that scans everything the server handles confirms that claim for the conversations tested. The limits are stated plainly: there is no forward secrecy, the server still sees metadata, trust on first use is only as strong as the first comparison, the thread-per-client server saturates a core at about 200 receivers, and all measurements, including our

own security testing, come from one machine and the authors. The next steps are a ratchet, a multi-device evaluation over real networks, an `asyncio` server with the full feature set, and an independent review.

REFERENCES

[1] J. Postel, "Transmission Control Protocol," IETF, RFC 793, Sep. 1981.

[2] W. Eddy, Ed., "Transmission Control Protocol (TCP)," IETF, RFC 9293, Aug. 2022.

[3] R. Braden, Ed., "Requirements for Internet Hosts – Communication Layers," IETF, RFC 1122, Oct. 1989.

[4] V. Jacobson, "Congestion avoidance and control," in *Proc. ACM SIGCOMM*, 1988, pp. 314–329.

[5] M. Allman, V. Paxson, and E. Blanton, "TCP congestion control," IETF, RFC 5681, Sep. 2009.

[6] J. H. Saltzer, D. P. Reed, and D. D. Clark, "End-to-end arguments in system design," *ACM Trans. Comput. Syst.*, vol. 2, no. 4, pp. 277–288, 1984.

[7] W. R. Stevens, B. Fenner, and A. M. Rudoff, *UNIX Network Programming, Vol. 1: The Sockets Networking API*, 3rd ed. Addison-Wesley, 2003.

[8] A. S. Tanenbaum and D. J. Wetherall, *Computer Networks*, 5th ed. Pearson, 2011.

[9] J. F. Kurose and K. W. Ross, *Computer Networking: A Top-Down Approach*, 8th ed. Pearson, 2021.

[10] F. Yergeau, "UTF-8, a transformation format of ISO 10646," IETF, RFC 3629, Nov. 2003.

[11] V. S. Pai, P. Druschel, and W. Zwaenepoel, "Flash: An efficient and portable Web server," in *Proc. USENIX Annu. Tech. Conf.*, 1999.

[12] M. Welsh, D. Culler, and E. Brewer, "SEDA: An architecture for well-conditioned, scalable Internet services," in *Proc. ACM SOSP*, 2001, pp. 230–243.

[13] J. K. Ousterhout, "Why threads are a bad idea (for most purposes)," invited talk, USENIX Winter Tech. Conf., 1996.

[14] R. von Behren, J. Condit, and E. Brewer, "Why events are a bad idea (for high-concurrency servers)," in *Proc. HotOS IX*, 2003.

[15] E. A. Lee, "The problem with threads," *Computer*, vol. 39, no. 5, pp. 33–42, May 2006.

[16] M. Krohn, E. Kohler, and M. F. Kaashoek, "Events can make sense," in *Proc. USENIX Annu. Tech. Conf.*, 2007.

[17] D. Kegel, "The C10K problem," 1999. [Online]. Available: http://www.kegel.com/c10k.html

[18] J. Oikarinen and D. Reed, "Internet Relay Chat protocol," IETF, RFC 1459, May 1993.

[19] C. Kalt, "Internet Relay Chat: Client protocol," IETF, RFC 2812, Apr. 2000.

[20] P. Saint-Andre, "Extensible Messaging and Presence Protocol (XMPP): Core," IETF, RFC 6120, Mar. 2011.

[21] P. Saint-Andre, "Extensible Messaging and Presence Protocol (XMPP): Instant messaging and presence," IETF, RFC 6121, Mar. 2011.

[22] I. Fette and A. Melnikov, "The WebSocket protocol," IETF, RFC 6455, Dec. 2011.

[23] OASIS, "MQTT Version 5.0," OASIS Standard, Mar. 2019.

[24] S. Deering, "Host extensions for IP multicasting," IETF, RFC 1112, Aug. 1989.

[25] P. Th. Eugster, P. A. Felber, R. Guerraoui, and A.-M. Kermarrec, "The many faces of publish/subscribe," *ACM Comput. Surv.*, vol. 35, no. 2, pp. 114–131, Jun. 2003.

[26] K. P. Birman and T. A. Joseph, "Reliable communication in the presence of failures," *ACM Trans. Comput. Syst.*, vol. 5, no. 1, pp. 47–76, 1987.

[27] K. P. Birman, "The process group approach to reliable distributed computing," *Commun. ACM*, vol. 36, no. 12, pp. 37–53, Dec. 1993.

[28] E. Rescorla, "The Transport Layer Security (TLS) protocol version 1.3," IETF, RFC 8446, Aug. 2018.

[29] N. Unger *et al.*, "SoK: Secure messaging," in *Proc. IEEE Symp. Security and Privacy*, 2015, pp. 232–249.

[30] K. Cohn-Gordon, C. Cremers, B. Dowling, L. Garratt, and D. Stebila, "A formal security analysis of the Signal messaging protocol," in *Proc. IEEE Eur. Symp. Security and Privacy*, 2017, pp. 451–466.

[31] M. Marlinspike and T. Perrin, "The Double Ratchet algorithm," Signal, Rev. 1, Nov. 2016.

[32] Python Software Foundation, "socket," *Python 3 Documentation*. https://docs.python.org/3/library/socket.html

[33] Python Software Foundation, "threading," *Python 3 Documentation*. https://docs.python.org/3/library/threading.html

[34] Python Software Foundation, "struct," *Python 3 Documentation*. https://docs.python.org/3/library/struct.html

[35] Python Software Foundation, "ssl," *Python 3 Documentation*. https://docs.python.org/3/library/ssl.html

[36] The Matrix.org Foundation, "Matrix Specification," https://spec.matrix.org, accessed 2026.

[37] XMPP Standards Foundation, "XEP-0384: OMEMO Encryption," https://xmpp.org/extensions/xep-0384.html, accessed 2026.

[38] Python Software Foundation, "asyncio," *Python 3 Documentation*. https://docs.python.org/3/library/asyncio.html

[39] J. Postel and J. Reynolds, "File Transfer Protocol (FTP)," IETF, RFC 959, Oct. 1985.

[40] T. Ylonen and C. Lonvick, Ed., "The Secure Shell (SSH) protocol architecture," IETF, RFC 4251, Jan. 2006.

[41] S. Cheshire and M. Krochmal, "Multicast DNS," IETF, RFC 6762, Feb. 2013.

[42] S. Cheshire and M. Krochmal, "DNS-based service discovery," IETF, RFC 6763, Feb. 2013.

[43] C. Percival and S. Josefsson, "The scrypt password-based key derivation function," IETF, RFC 7914, Aug. 2016.

[44] A. Langley, M. Hamburg, and S. Turner, "Elliptic curves for security," IETF, RFC 7748, Jan. 2016.

[45] S. Josefsson and I. Liusvaara, "Edwards-curve digital signature algorithm (EdDSA)," IETF, RFC 8032, Jan. 2017.

[46] Y. Nir and A. Langley, "ChaCha20 and Poly1305 for IETF protocols," IETF, RFC 8439, Jun. 2018.

[47] H. Krawczyk and P. Eronen, "HMAC-based extract-and-expand key derivation function (HKDF)," IETF, RFC 5869, May 2010.

[48] R. Barnes, B. Beurdouche, R. Robert, J. Millican, E. Omara, and K. Cohn-Gordon, "The Messaging Layer Security (MLS) protocol," IETF, RFC 9420, Jul. 2023.

[49] M. Marlinspike and T. Perrin, "The X3DH key agreement protocol," Signal, Rev. 1, Nov. 2016.

[50] P. A. Grassi *et al.*, "Digital identity guidelines: Authentication and lifecycle management," NIST Special Publication 800-63B, Jun. 2017.

[51] B. Schneier and J. Kelsey, "Secure audit logs to support computer forensics," *ACM Trans. Inf. Syst. Secur.*, vol. 2, no. 2, pp. 159–176, May 1999.

[52] S. A. Crosby and D. S. Wallach, "Efficient data structures for tamper-evident logging," in *Proc. USENIX Security Symp.*, 2009, pp. 317–334.

[53] Python Software Foundation, "sqlite3," *Python 3 Documentation*. https://docs.python.org/3/library/sqlite3.html

[54] Python Software Foundation, "hashlib," *Python 3 Documentation*. https://docs.python.org/3/library/hashlib.html

[55] H. Goud B, R. R. Lingala, and S. Manocha, "LANChat-H: source code, tests and benchmark scripts," GitHub repository, 2026. [Online]. Available: https://github.com/revanthreddy0906/LAN-Chat-Application